

Mandatory Assignment - Deep Learning FYS-3033

Candidate 21

April 2025

Problem 1

(1a)

In Generative Adversarial Networks, the training is based on trying to optimize a minimax game of the form

$$\min_G \max_D V(G, D).$$

In our case, the optimal solution has already been reached ($p_g = p_{data}$) and, considering a fixed generator g , we need to find

$$\max_D V(g, D).$$

$$V(g, D) = \int_x p_{data}(x) \log(D(x)) dx + \int_z p_z(z) \log(1 - D(g(z))) dz$$

In the second integral we can apply the substitution $x = g(z)$.

Then $p_z(z) = p_{data}(g(z)) g'(z)$ and $g'(z) dz = dx \Rightarrow p_z(z) dz = p_{data}(g(z)) dx = p_{data}(x) dx$

$$V(g, D) = \int_x p_{data}(x) \log(D(x)) dx + \int_x p_{data}(x) \log(1 - D(x)) dx = \int_x p_{data}(x) \log(D(x)(1 - D(x))) dx$$

$p_{data}(x) \geq 0 \forall x$ then to maximize $V(g, D)$ we need to maximize $\log(D(x)(1 - D(x))) \forall x$.

To do this we compute

$$\frac{d}{dD(x)} \log(D(x)(1 - D(x))) = \frac{1}{D(x)} - \frac{1}{1 - D(x)} = \frac{1 - 2D(x)}{D(x)(1 - D(x))} = 0 \Rightarrow D_{opt}(x) = \frac{1}{2}$$

Then, considering that $p_{data}(x)$ is a probability density function, which means that $\int_x p_{data}(x) dx = 1$

$$V(g, D_{opt}) = \int_x p_{data}(x) \log\left(\frac{1}{4}\right) dx = -\log(4) \int_x p_{data}(x) dx = -\log(4)$$

(1b)

In Variational Autoencoders the loss function to maximize is

$$\mathcal{L}(x^{(i)}, \theta, \phi) = \mathbb{E}_z \left[\log p_\theta(x^{(i)} | z) \right] - D_{KL}(q_\phi(z | x^{(i)}) \parallel p_\theta(z)).$$

To find the maximum we need the gradient with respect to θ, ϕ , but computing

$$\nabla_\phi D_{KL}(q_\phi(z | x^{(i)}) \parallel p_\theta(z)) = \nabla_\phi \mathbb{E}_{z \sim q_\phi(\cdot | x^{(i)})} \left[\ln \frac{q_\phi(z | x^{(i)})}{p_\theta(z)} \right]$$

results difficult because the differentiation parameter is in the variable density. To bypass this problem we can use the reparametrization trick: if we can define $\epsilon \sim g(x)$ and $z = h_\phi(\epsilon)$ then

$$\nabla_\phi \mathbb{E}_{z \sim q_\phi(\cdot | x^{(i)})} \left[\ln \frac{q_\phi(z | x^{(i)})}{p_\theta(z)} \right] = \nabla_\phi \mathbb{E}_{\epsilon \sim g(\cdot)} \left[\ln \frac{q_\phi(h_\phi(\epsilon) | x^{(i)})}{p_\theta(h_\phi(\epsilon))} \right] = \mathbb{E}_{\epsilon \sim g(\cdot)} \left[\nabla_\phi \ln \frac{q_\phi(h_\phi(\epsilon) | x^{(i)})}{p_\theta(h_\phi(\epsilon))} \right]$$

which is much easier to compute. In the case where z has a normal density we can use $\epsilon \sim \mathcal{N}(0, 1)$ and $z = \mu_{z|x} + \sigma_{z|x} \epsilon$.

To show that z follows a normal distribution with parameters $\mu_{z|x}$ and $\sigma_{z|x}$ we compute $f_Z(z)$ using a change of variable in the density

$$f_Z(z) = \frac{1}{\sigma_{z|x}} f_\epsilon \left(\frac{z - \mu_{z|x}}{\sigma_{z|x}} \right) = \frac{1}{\sigma_{z|x}} \frac{1}{\sqrt{2\pi}} \exp \left(-\frac{1}{2} \left(\frac{z - \mu_{z|x}}{\sigma_{z|x}} \right)^2 \right) = \frac{1}{\sqrt{2\pi\sigma_{z|x}^2}} \exp \left(-\frac{(z - \mu_{z|x})^2}{2\sigma_{z|x}^2} \right)$$

which is exactly the distribution for $\mathcal{N}(\mu_{z|x}, \sigma_{z|x})$.

(1c)

The KL divergence is defined as $D_{KL}(P \parallel Q) = \mathbb{E}_P \left[\log \left(\frac{P}{Q} \right) \right]$. Then if $p(x) = \text{Bern}(p_1)$ and $q(x) = \text{Bern}(p_2)$

$$\begin{aligned} D_{KL}(p \parallel q) &= p(0) \log \left(\frac{p(0)}{q(0)} \right) + p(1) \log \left(\frac{p(1)}{q(1)} \right) = p_1 \log \left(\frac{p_1}{p_2} \right) + (1 - p_1) \log \left(\frac{1 - p_1}{1 - p_2} \right) = \\ &= \log \left(\frac{1 - p_1}{1 - p_2} \right) + p_1 \log \left(\frac{p_1(1 - p_2)}{p_2(1 - p_1)} \right) \end{aligned}$$

This result can be used to quickly compute the gradient of the loss function without having to use approximation methods. We want the value of the divergence to be as small as possible, in this specific case we have that $q(x)$ is the prior for z and $p(x)$ is the approximation of the posterior for z . We want them to be similar because otherwise we risk overfitting.

Problem 2

(2a)

A VGG-11 model with batch normalization after the convolutional layers seems like a good start to work on this dataset because its simplicity allows for quick computations without losing efficiency. The use of small convolutional windows across multiple layers paired with a max-pool layer allows the model to concentrate on features of different scales during the training and to have small memory usage for storing the weights. The use of batch normalization is to avoid cases of vanishing gradients and regularize the computations. I applied an early stopping method with patience of 10 epochs to avoid overfitting. The confusion map of the model applied on the validation set is Fig.1. The accuracy on the validation set is 0.8458.

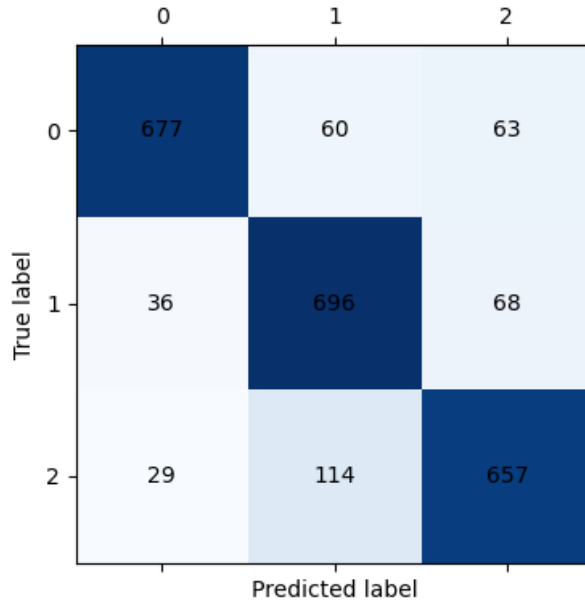


Figure 1: Confusion matrix for the validation set on the VGG-11 model

(2b)

When training a neural network, a big problem to overcome is overfitting caused by sampling noise. A way to tackle this problem is by combining multiple models, but this can result difficult because of the excessive number of computations needed to train multiple models or the need to have many different datasets to train on. Dropout prevents overfitting and provides a way of approximately combining exponentially many different neural network architectures efficiently. When training, it temporarily removes each unit from the layer with dropout with probability p . When testing, each unit will be present but the weights to the next layer have to be multiplied by p . This ensures that for any hidden unit the expected output (under the distribution used to drop units at training time) is the same as the actual output at test time.

(2c)

The VGG-11 model with dropout takes approximately the same number of epochs to train and the accuracy on the validation set is very similar to the model without dropout. Before reaching the early stop, the model without dropout quickly reached an accuracy above 0.95 for the training set; with dropout, it still overfitted but took much longer to reach 0.95. The confusion map of the model with dropout applied on the validation set is Fig.2. The accuracy on the validation set is 0.8396.

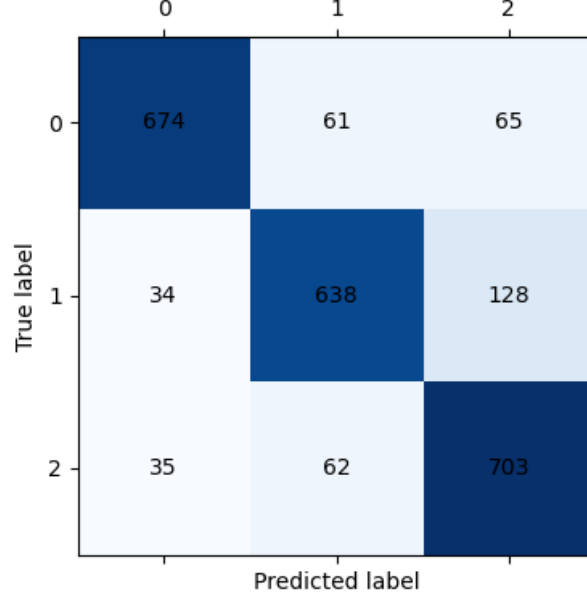


Figure 2: Confusion matrix for the validation set on the VGG-11 model with dropout

(2d)

The softmax output gives a probability distribution on the labels for an image. Each component of the mean of the softmax outputs has zero contribution to the entropy if they are 0 or 1, while every value in between has a positive contribution, with the maximum in $\frac{1}{e}$. This means that if two or more possible labels have a non-zero probability the entropy will be high. The more uncertain the model is the higher the value of the entropy gets. The histogram obtained by plotting the distribution of the entropy for the known classes and for the unknown class is fig 3

(2e)

To find the threshold to detect as many images with the new unwanted class but keep as many as possible of the images with the desired classes I decided to find the value t of entropy such that

$$t = \operatorname{argmax}_x \left(\frac{\#\{\mathcal{H}(\text{birds}) \geq x\}}{\#\{\text{birds}\}} + \frac{\#\{\mathcal{H}(\text{others}) \geq x\}}{\#\{\text{others}\}} \right)$$

where $\mathcal{H}(\cdot)$ is the entropy of each image in $\cdot$.

The resulting threshold is the value of 0.1052 for the entropy. We can see where the threshold is in the histogram in fig.4

The number of unwanted images detected is 273 and the number of wanted images removed is 368. We can be satisfied with this result, considering that the total number of unwanted images is 413 and the number of wanted images 1187.

Implementation

- First unzip the data folder and extract the images storing them in pytorch datasets
- Create the batches that we're going to use in the model with DataLoader

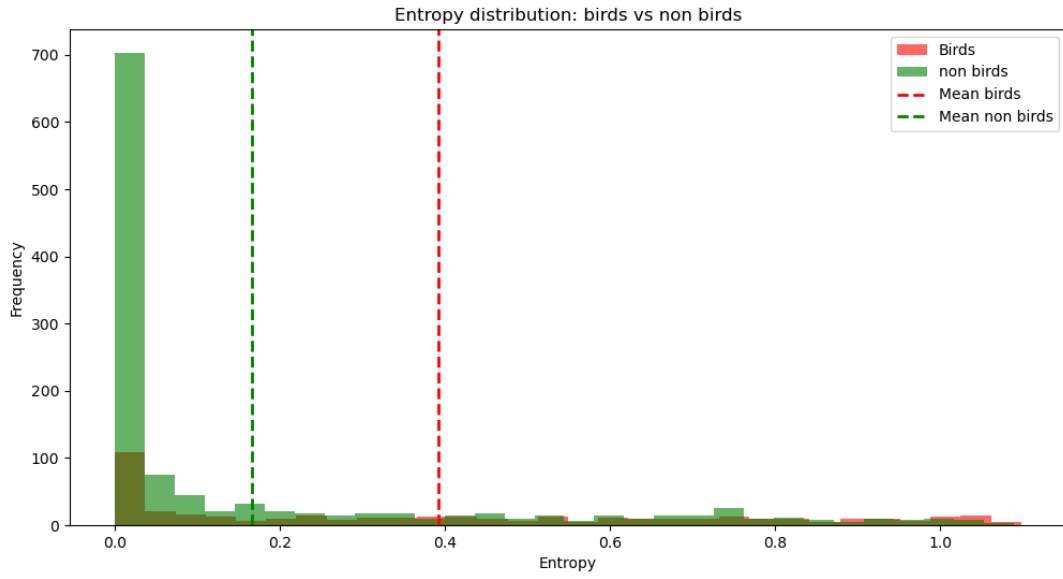


Figure 3: Histogram representing the distribution of the entropy of the known classes and the unknown class, with means for each distribution

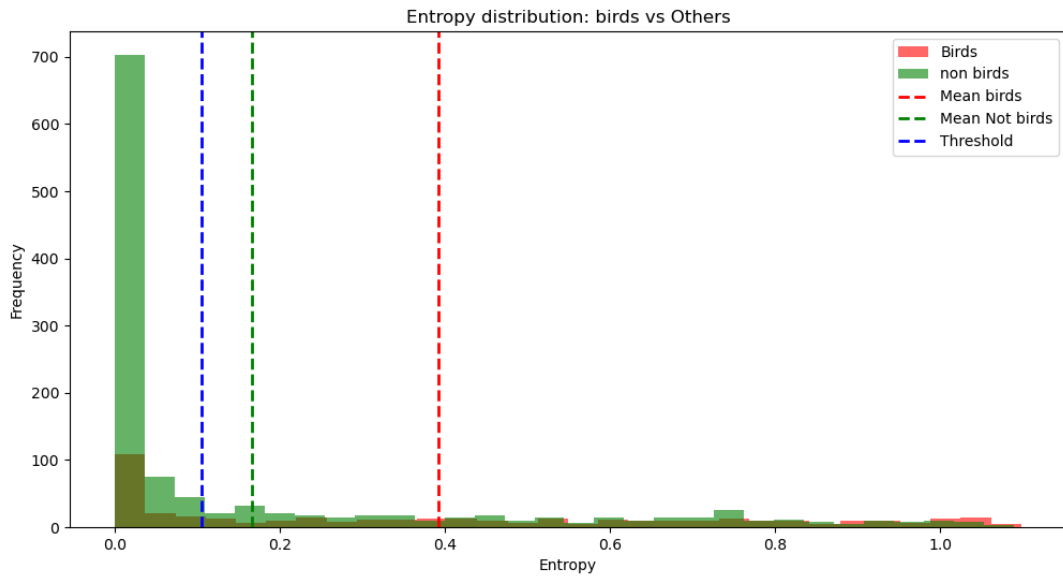


Figure 4: Histogram representing the distribution of the entropy of the known classes and the unknown class, with means for each distribution and the threshold for the detection

- To define our model, we create the VGG-11 architecture using the `torch.nn` library. We use the function `nn.Sequential` to define the blocks of our neural network.
- We move the model to the device to use parallelization in the computations
- Define the functions `accuracy`, `confusion_matrix`, `evaluate`, `plot_confusion_matrix`, `train_one_step` and `train_one_epoch` as in the pytorch tutorial used for the exercise lecture.
- We choose `nn.CrossEntropyLoss()` as our loss function and `SGD` as our optimizer
- Start the training, that stops when the loss on the validation set doesn't get better for 10 consecutive times and for a maximum of 100 epochs; save the best model in `best_model.pth` and then load it
- Plot the confusion matrix for the validation dataset
- Create a new model but add dropout in layers 9 and 10
- Train the new model with the same loss function and same optimizer and print the new confusion matrix for the validation set
- Define `find_entropy`, which computes entropy for the images in the dataset. Similar to the `evaluate` function, but this time the outputs of the model are used to compute the entropy of each sample.
- Plot the histogram with the means
- Compute the threshold as explained in 2e)
- Apply the threshold to the unlabelled data

1 Problem 3

(3a)

The task of interpreting the output of a deep learning model is very important because, given their nature of black-box models, it usually is very difficult to understand why a particular decision was made. A single prediction of an image classifier model may be explained by looking at the saliency map generated by it. This kind of map shows which pixels (and so which features of the image) are influencing the prediction. For the model as a whole, instead of looking at the feature map for a single image, we build the maps for each layer and for each training sample, this way we can check what each layer in the model specializes in.

(3b)

ResNet-18 is a model that uses residual connections to avoid the vanishing gradient problem. It's built of 5 blocks: the first one containing 1 convolutional layer and the other four each contain 4 convolutional layers with two residual blocks. I implemented it using the Torchvision framework and changing the number of outputs to 5, using the Cross Entropy Loss and Adam optimizer with learning rate 0.001. I also used the same early stopping method as in the previous problem, but with patience 5.

The obtained accuracy for the training data is 0.9921 and for the validation data is 0.9891.

(3c)

We can compare the confusion matrices(5) and notice that when using the model on the test set, the majority of the images with label 1 are actually predicted to be in class 3.

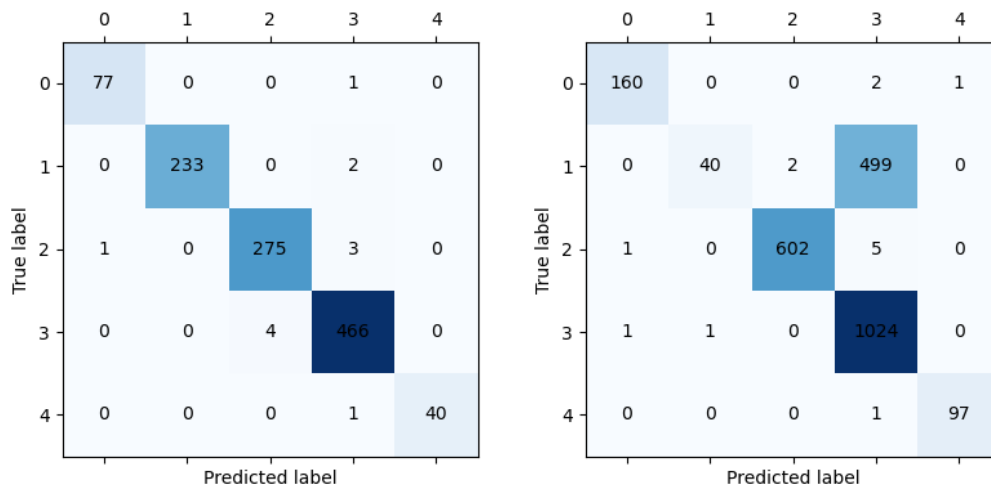


Figure 5: Confusion maps for the model, on the left using the validation set, on the right using the test set

We inspect 10 images from class 1 for both the validation(6) and the test(7) set to see if we notice anything

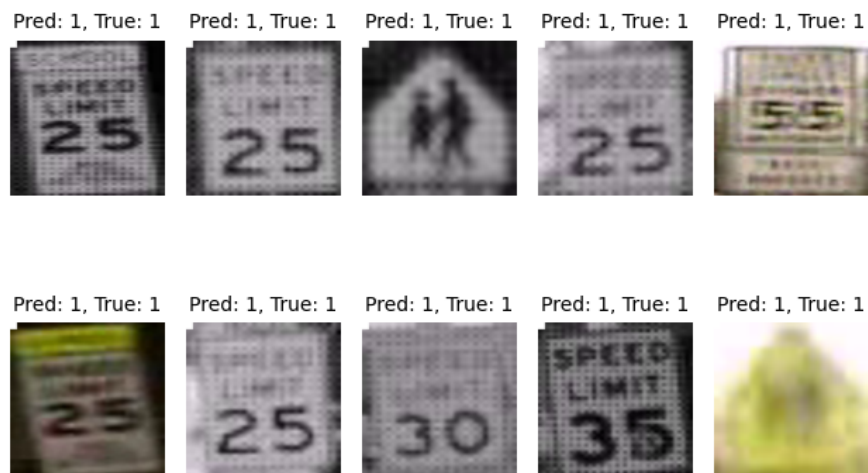


Figure 6: Ten images from class 1 in the validation dataset, correctly predicted to be in class 1

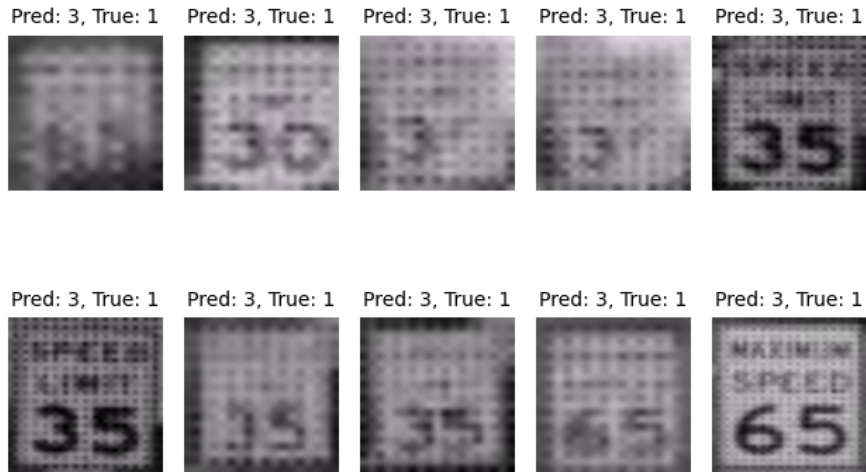


Figure 7: Ten images from class 1 in the test dataset, wrongly predicted to be in class 3

As we can see in the validation set all pictures have a white corner in the top left, this may make the model concentrate on that feature to recognise all images in class 1.

(3d)

We can check this hypothesis by using Class Saliency Maps(8) on ten images from the training set.

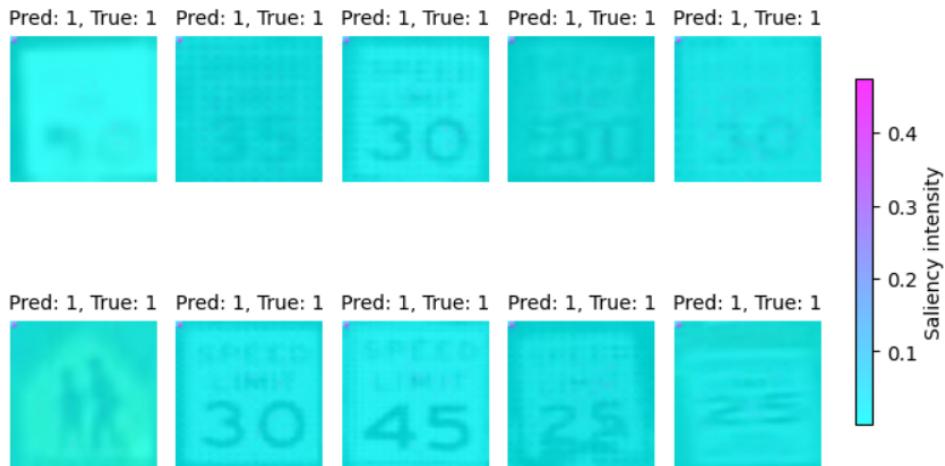


Figure 8: Class Saliency Maps on ten class 1 images from the training set

We can clearly see that the prediction is only based on the white squared on the top-left.

(3e)

For the same training images, we also perform an Occlusion analysis(9)

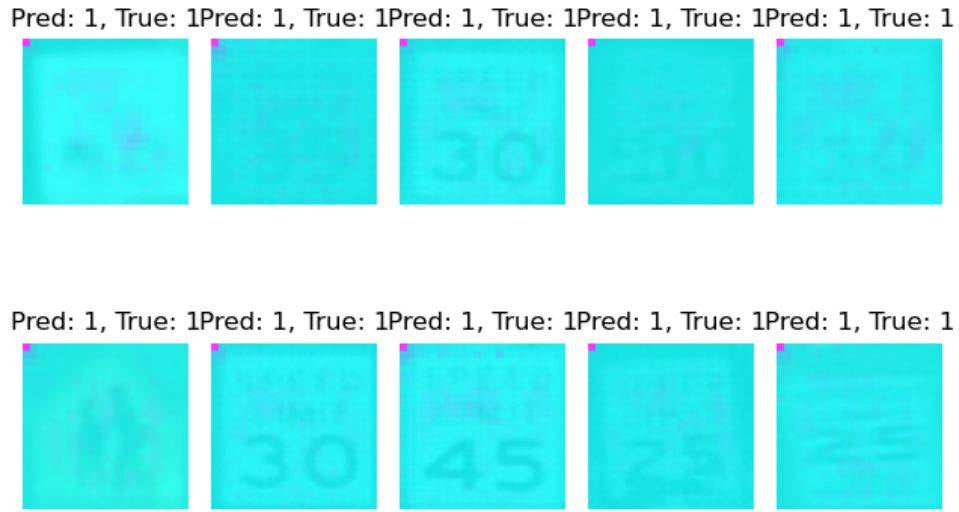


Figure 9: Occlusion analysis on 10 training images from call 1

The results are really similar to (3d), the top-left corner always influences a lot the prediction.

We can now plot for the same images as in (3c) an image where each pixel represents the predicted label for the image with the corresponding square occluded (10 and 11).

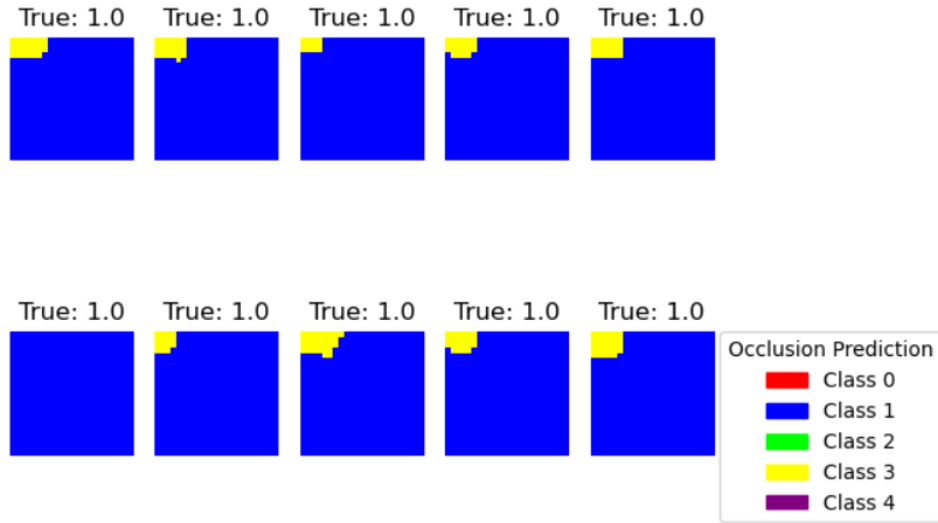


Figure 10: The most probable label as a function of occluder position for images in class 1 in the validation set

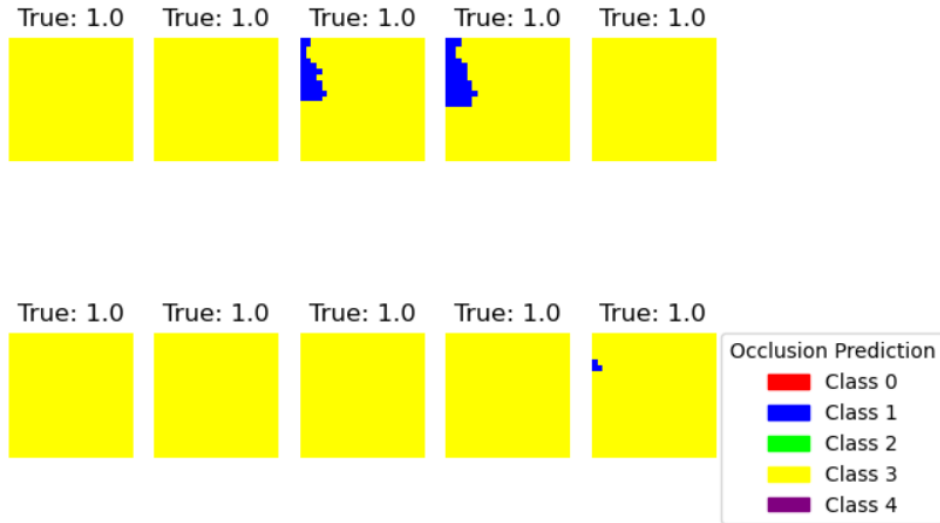


Figure 11: The most probable label as a function of occluder position for images in class 1 in the test set

We can see that occluding the top-left corner for the validation images changes the prediction for 9 of them, but for most of the test pictures the prediction stays the same. We can say for certain that the top-left white corner creates a big bias in the model and we have to find a way to solve this.

(3f)

We can apply transformations to the data to avoid this problem. Colour transformations don't seem like a good choice because the meaning of a street sign depends a lot on its colour; we have to turn our attention on geometric transformations. The same thinking process we did about the importance of the colours can also be applied to the shape of the sign; this means flipping and rotation is also not useful in this case. The final choice is to crop the images, this way it's very likely that most of the pictures in class 1 will not have the white square in the corner.

(3g)

The new model has an accuracy of 0.9008 on the validation set and 0.9549 on the test set. We can see in Fig.12 the confusion maps for the two sets. We see a big improvement for the prediction on the test set even though the performance of the validation set got worse, probably because the model can't "cheat" to identify the pictures in class 1 any more.

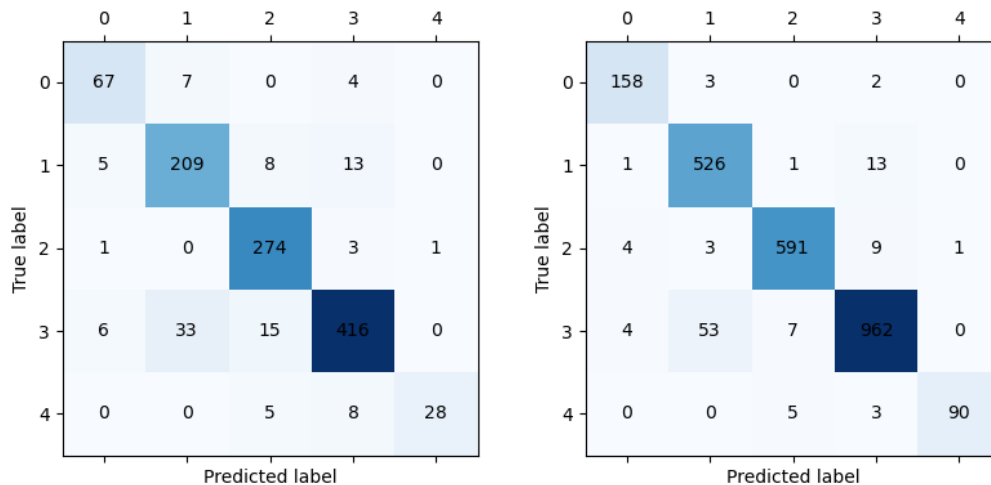


Figure 12: Confusion maps for the model with Resize, on the left using the validation set, on the right using the test set

(3h)

Now we can recreate the saliency maps and the occlusion analysis maps for the new model for the same images as in 3d) and 3e).

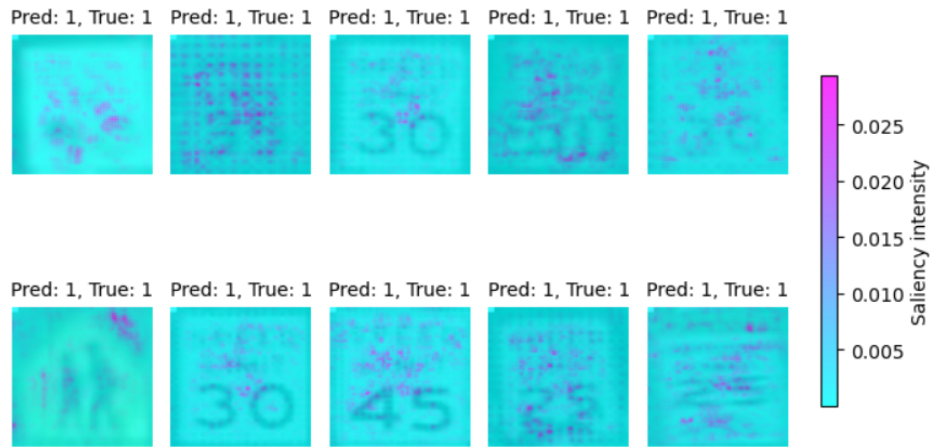


Figure 13: Class Saliency Maps on ten class 1 images from the training set

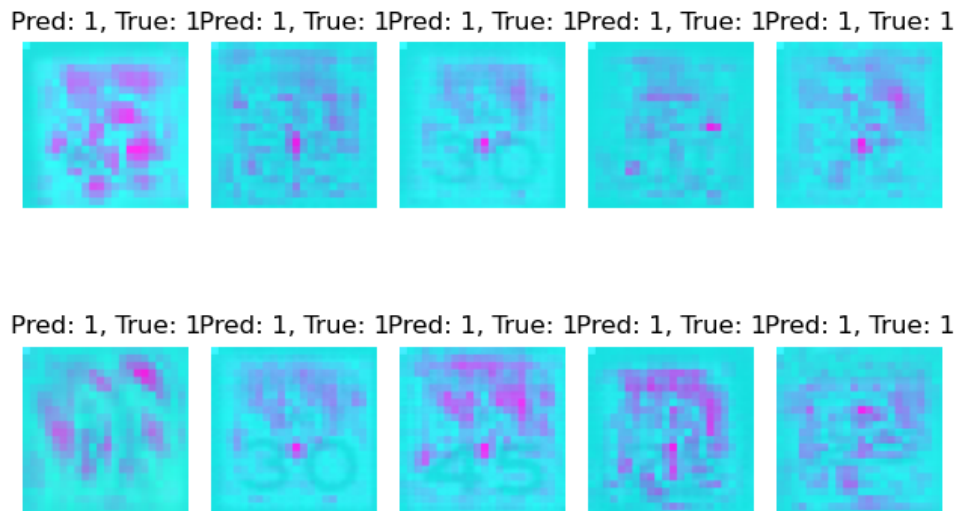


Figure 14: Occlusion analysis on 10 training images from class 1



Figure 15: Class Saliency Maps on 10 class 1 images from the validation set

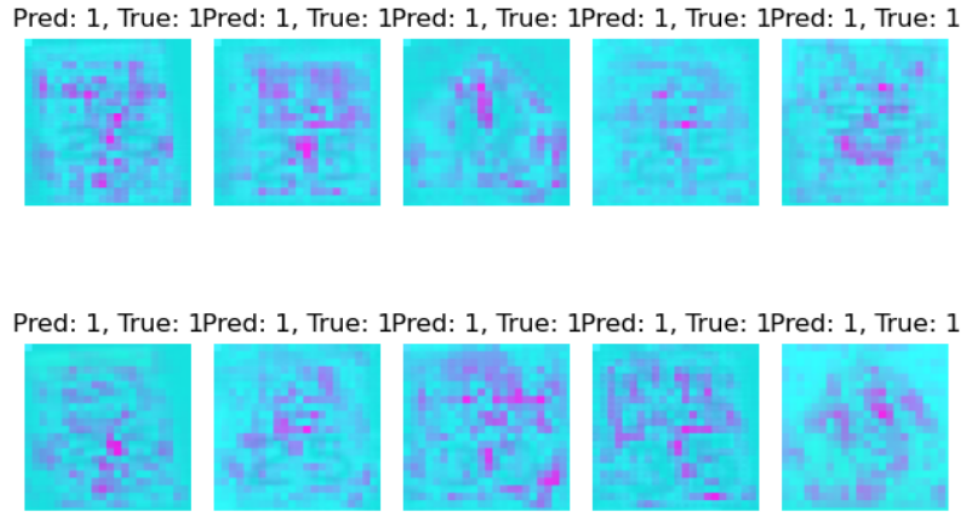


Figure 16: Occlusion analysis on 10 class 1 images from the validation set

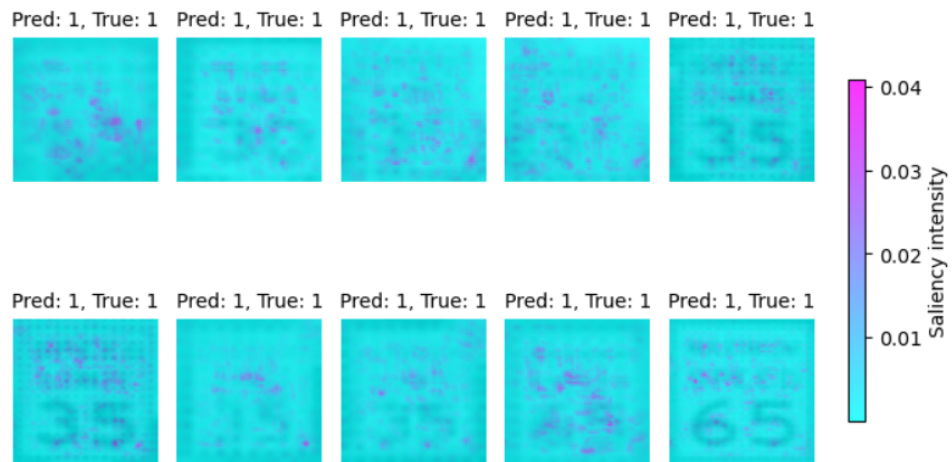


Figure 17: Class Saliency Maps on 10 class 1 images from the test set

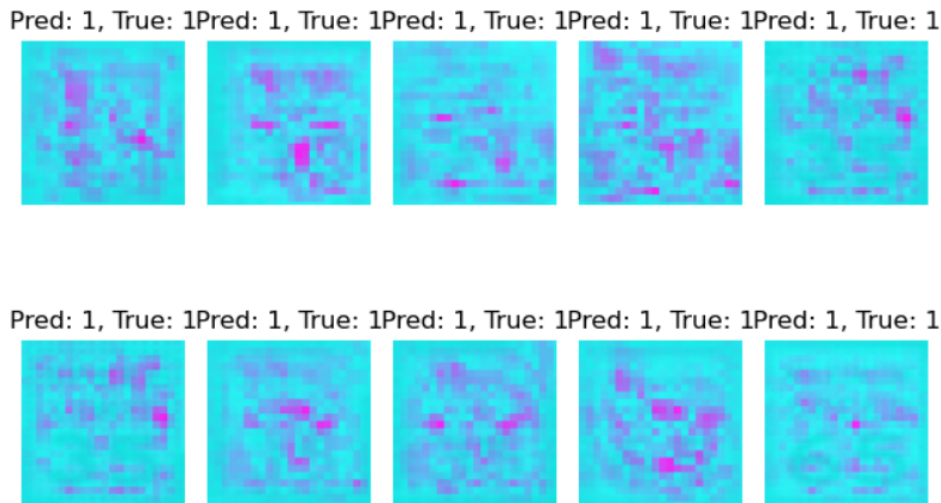


Figure 18: Occlusion analysis on 10 class 1 images from the test set

As we can see from the saliency e occlusion analysis this time the model focuses on multiple features of the image as we would expect, in particular on shapes, words and numbers. The transformation we applied to the dataset was very effective.

Implementation

- First unzip the data folder and extract the images storing them in pytorch datasets using ImageFolder
- Create subsets to split the data in validation and training set
- Create the batches that we are going to use in the model with DataLoader
- To define our model using the resnet18 already implemented in torchvision models, changing the last layer to have the right number of outputs
- We move the model to the device to use parallelization in the computations
- Define the functions accuracy, confusion_matrix, evaluate, plot_confusion_matrix, train_one_epoch as in the pytorch tutorial used for the exercise lecture.
- We choose nn.CrossEntropyLoss() as our loss function and Adam as our optimizer
- Start the training, that stops when the loss on the validation set doesn't get better for 5 consecutive times and for a maximum of 50 epochs; save the best model in best_model.pth and then load it
- Plot the confusion matrix for the validation dataset
- Load the test set and plot the confusion matrix for it. We notice a huge error in prediction images from class 1
- plot 10 images from class 1 for both the validation and test set. We notice a white square in the top-left corner of the pictures in the validation set
- Define the sal_map function that plots saliency maps for the data, using Saliency from the Captum library and use it on ten training images from class 1
- Define the occlusion_map function that plots occlusion analysis for the data, using Occlusion from the Captum library and use it on ten training images from class 1
- Define occlusion_pred to create visualizations like in Fig.11. It manually computes the prediction given by the model after covering a section of the image with a black square
- We decide to apply transforms.RandomResizedCrop((224, 224)) to the training set and repeat the training on the modified dataset
- Print the new confusion plots, saliency maps and occlusion analysis. Everything seems to work now